

Software Requirements Specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Booking Platform REST API. The system allows authenticated users to manage services and allows customers to create and manage service bookings through HTTP endpoints.

1.2 Scope

The Booking Platform REST API provides:

- User registration, login, and token refresh
- Authenticated service management
- Public booking creation
- Authenticated booking administration
- Validation and business rule enforcement for booking workflows
- Swagger and Postman based API documentation

This system is intended for use as a backend service for a booking platform where service providers manage offerings and customers submit booking requests.

1.3 Definitions and Acronyms

- API: Application Programming Interface
- JWT: JSON Web Token
- REST: Representational State Transfer
- SRS: Software Requirements Specification
- UUID: Universally Unique Identifier

1.4 References

- Project README
- Swagger documentation exposed at `/api/docs`

- Postman collection in postman/

2. Overall Description

2.1 Product Perspective

The application is a standalone REST API built with NestJS, TypeScript, TypeORM, and PostgreSQL. It exposes HTTP endpoints for authentication, service management, and booking management.

2.2 Product Functions

The system shall support the following major functions:

- Register new users
- Authenticate existing users and issue JWTs
- Refresh authentication tokens
- Create, list, retrieve, update, and delete services
- Create bookings without authentication
- List, retrieve, update status of, and cancel bookings with authentication
- Enforce booking validation and lifecycle rules

2.3 User Classes and Characteristics

- Service Manager Uses authenticated endpoints to manage services and bookings.
- Customer Uses the public booking endpoint to create a booking.

2.4 Operating Environment

- Node.js runtime
- NestJS application server
- PostgreSQL database
- HTTP clients such as Swagger UI, Postman, frontend apps, or curl

2.5 Assumptions and Dependencies

- PostgreSQL is available and correctly configured through environment variables.
- Database migrations are executed before using the API in a new environment.

- JWT secrets are configured through environment variables.
- Booking datetime validation uses server time.

3. External Interface Requirements

3.1 API Interface

Base URL:

- <http://localhost:3000>

Documentation:

- Swagger UI at </api/docs>

Authentication:

- Bearer token authentication for protected endpoints

3.2 Software Interfaces

- PostgreSQL for persistence
- TypeORM for ORM and migrations
- Swagger/OpenAPI for API documentation
- Postman collection for manual API testing

4. System Features and Functional Requirements

4.1 Authentication

4.1.1 User Registration

- The system shall provide POST </auth/register>.
- The system shall accept email, password, and optional name.
- The system shall validate email format.
- The system shall validate password strength.
- The system shall reject duplicate email registrations.
- The system shall hash user passwords before persistence.
- The system shall return the created user and a token pair.

4.1.2 User Login

- The system shall provide POST /auth/login.
- The system shall validate user credentials.
- The system shall return an access token and a refresh token on successful login.

4.1.3 Token Refresh

- The system shall provide POST /auth/refresh.
- The system shall accept a refresh token.
- The system shall verify the refresh token and rotate it after success.

4.2 Service Management

4.2.1 Authorization

- The system shall require authentication for all service endpoints.

4.2.2 Create Service

- The system shall provide POST /services.
- The system shall create a service linked to the authenticated user.
- The system shall validate title, description, duration, price, and optional active flag.

4.2.3 List Services

- The system shall provide GET /services.
- The system shall support optional pagination using page and limit.
- The system shall support optional filtering by isActive.

4.2.4 Retrieve Service

- The system shall provide GET /services/:id.
- The system shall return 404 if the service does not exist.

4.2.5 Update Service

- The system shall provide PATCH /services/:id.
- The system shall update an existing service.
- The system shall return 404 if the service does not exist.

4.2.6 Delete Service

- The system shall provide DELETE /services/:id.
- The system shall return 404 if the service does not exist.
- The system shall prevent deletion when pending bookings exist for that service.

4.3 Booking Management

4.3.1 Public Booking Creation

- The system shall provide POST /bookings.
- The system shall allow booking creation without authentication.
- The system shall validate customer name, email, phone, service ID, booking date, and booking time.

4.3.2 Authenticated Booking Administration

- The system shall require authentication for:
 - GET /bookings
 - GET /bookings/:id
 - PATCH /bookings/:id/status
 - PATCH /bookings/:id/cancel

4.3.3 List Bookings

- The system shall provide GET /bookings.
- The system shall support pagination with page and limit.
- The system shall support filtering by booking status.
- The system shall support text search against customer name and customer email.

4.3.4 Retrieve Booking

- The system shall provide GET /bookings/:id.
- The system shall return 404 if the booking does not exist.

4.3.5 Update Booking Status

- The system shall provide PATCH /bookings/:id/status.

- The system shall allow only the following transitions:
 - PENDING -> CONFIRMED
 - PENDING -> CANCELLED
 - CONFIRMED -> COMPLETED
 - CONFIRMED -> CANCELLED
- The system shall reject all transitions from CANCELLED.
- The system shall reject all transitions from COMPLETED.

4.3.6 Cancel Booking

- The system shall provide PATCH /bookings/:id/cancel.
- The system shall reject cancellation of completed bookings.

5. Business Rules

- A booking must belong to an existing active service.
- Booking dates and times cannot be in the past.
- Duplicate bookings for the same service, date, and time shall be rejected while an existing booking is not cancelled.
- Cancelled bookings cannot be marked as completed.
- Only authenticated users can manage services.
- Customers can create bookings without authentication.

6. Data Requirements

6.1 User Entity

- id: UUID, primary key
- email: unique string
- password: hashed string
- name: optional string
- refreshToken: optional hashed string
- createdAt: timestamp

- updatedAt: timestamp

6.2 Service Entity

- id: UUID, primary key
- title: string
- description: text
- duration: integer in minutes
- price: decimal(10,2)
- isActive: boolean
- createdById: UUID referencing user
- createdAt: timestamp
- updatedAt: timestamp

6.3 Booking Entity

- id: UUID, primary key
- customerName: string
- customerEmail: string
- customerPhone: string
- serviceId: UUID referencing service
- bookingDate: date
- bookingTime: time
- status: enum with PENDING, CONFIRMED, CANCELLED, COMPLETED
- notes: optional text
- createdAt: timestamp
- updatedAt: timestamp

6.4 Relationships

- One user can create many services.
- One service can have many bookings.

- One booking belongs to one service.

7. Validation and Error Handling Requirements

- The system shall enforce request validation using a global validation pipeline.
- The system shall reject unknown properties in validated DTOs.
- The system shall transform request inputs into the expected types where applicable.
- The system shall return consistent error responses containing:
 - statusCode
 - message
 - error
 - timestamp
 - path

8. Non-Functional Requirements

8.1 Security

- Passwords shall be hashed before storage.
- Protected endpoints shall require JWT bearer tokens.
- Refresh tokens shall be stored in hashed form.

8.2 Reliability

- The system shall use migrations for schema setup.
- The API shall fail gracefully with structured error responses.

8.3 Maintainability

- The application shall use modular NestJS architecture.
- DTO-based validation shall be used for request contracts.
- API documentation shall be maintained in Swagger and Postman.

8.4 Performance

- List endpoints shall support pagination.

- Search and filter operations shall be executed in the database layer.

8.5 Usability

- Swagger UI shall be available for manual exploration.
- A Postman collection shall be provided for testing the full API flow.

9. Constraints

- The backend shall use NestJS and TypeScript.
- The persistence layer shall use PostgreSQL.
- The ORM layer shall use TypeORM.
- The authentication model shall use JWT.

10. Acceptance Criteria

- A user can register, log in, and refresh tokens successfully.
- An unauthenticated request to service management endpoints is rejected.
- An authenticated user can create, list, retrieve, update, and delete services.
- A public user can create a booking for an active service.
- A booking for a missing or inactive service is rejected.
- A booking in the past is rejected.
- A duplicate slot booking is rejected while the existing slot is not cancelled.
- A cancelled booking cannot transition to completed.
- Swagger documentation and Postman artifacts are available.

11. Future Enhancements

- Role-based authorization
- Rate limiting
- Notifications for booking lifecycle events
- Audit logs
- Soft delete support

- Extended automated test coverage